

OBJECT ORIENTED PROGRAMMING

CSL-210

Supervisor Name: Mam Nida Khalid

Submission Date: 13th May 2025

SNAKE GAME (Report)



NAME: MUHAMMAD UMAR HAMMAD

CLASS: BS (CS)-2

ENROLLMENT N.O: 09-134242-032

NAME: ABDULLAH KHAN

CLASS: BS (CS)-2

ENROLLMENT N.O: 09-134242-003

Department of Computer Science

◆ 1. Introduction

- Brief overview of the project: This project is a classic Snake Game developed using C++ and SFML (Simple and Fast Multimedia Library). The game allows the player to control a snake that moves around the screen, eats food to grow longer, and ends if the snake collides with itself or the wall.
 - What problem it solves: The project showcases Object-Oriented Programming (OOP) principles in action and demonstrates the application of inheritance, encapsulation, abstraction, and polymorphism in a graphical game environment. It also provides an engaging way to understand basic game development using C++.
 - Objective of the project: The objective is to build a functional and interactive snake game using C++ and SFML while implementing and demonstrating core OOP concepts.
-

◆ 3. Tools & Technologies Used

- C++ version: C++17
 - IDE: Code::Blocks
 - Compiler: g++ (GNU Compiler Collection)
 - Libraries: SFML (Simple and Fast Multimedia Library)
-

◆ 4. System Requirements

- Hardware requirements:
 - Minimum 2 GB RAM
 - Dual-core CPU
 - Software requirements:
 - Operating System: Windows 10/Linux
 - IDE: Code::Blocks with SFML configured
 - SFML 2.5 or above
-

◆ 5. System Design

- Explanation of the structure: The game is built using the OOP paradigm. Classes are designed for Game, Snake, Food, and SnakeSegment. The GameObject base class is used to implement inheritance and polymorphism.
 - Class diagrams:
 - GameObject (abstract base class)
 - virtual void draw(sf::RenderWindow&)
 - virtual sf::Vector2f getPosition()
 - Snake (inherits GameObject)
 - Food (inherits GameObject)
 - Game (main controller)
 - Flowcharts or logic flow:
 - Game Loop → Event Handling → Snake Movement → Collision Detection → Food Consumption → Rendering
-

◆ 6. Implementation

- Code Description:
 - GameObject: Abstract base class to enforce common interface.
 - Snake: Manages body segments, movement, collision, and growth.
 - Food: Manages spawning and drawing of food object.
 - Game: Main class handling game loop, input, and logic.

1. **GameObject: Abstract Base Class:**

```
class GameObject {  
public:  
    virtual void draw(sf::RenderWindow& window) = 0;  
    virtual sf::Vector2f getPosition() const = 0;  
    virtual ~GameObject() {}  
};
```

2. SnakeSegment: Represents a Snake Segment

```
class SnakeSegment : public GameObject {
private:
    sf::RectangleShape shape;

public:
    SnakeSegment(sf::Vector2f position) {
        shape.setSize(sf::Vector2f(GRID_SIZE, GRID_SIZE));
        shape.setFillColor(sf::Color::Green);
        shape.setPosition(position);
    }

    void setPosition(sf::Vector2f pos) {
        shape.setPosition(pos);
    }

    sf::Vector2f getPosition() const override {
        return shape.getPosition();
    }

    void draw(sf::RenderWindow& window) override {
        window.draw(shape);
    }
};
```

3. Food: Manages Food Spawning and Drawing

```
// ----- Snake Segment -----  
class SnakeSegment : public GameObject {  
private:  
    sf::RectangleShape shape;  
  
public:  
    SnakeSegment(sf::Vector2f position) {  
        shape.setSize(sf::Vector2f(GRID_SIZE, GRID_SIZE));  
        shape.setFillColor(sf::Color::Green);  
        shape.setPosition(position);  
    }  
  
    void setPosition(sf::Vector2f pos) {  
        shape.setPosition(pos);  
    }  
  
    sf::Vector2f getPosition() const override {  
        return shape.getPosition();  
    }  
  
    void draw(sf::RenderWindow& window) override {  
        window.draw(shape);  
    }  
};
```

4. Snake: Manages Snake Behavior

```
// ----- Snake -----
class Snake {
private:
    std::vector<SnakeSegment> body;
    sf::Vector2f direction;

public:
    Snake() {
        body.emplace_back(sf::Vector2f(WINDOW_WIDTH / 2, WINDOW_HEIGHT / 2));
        direction = sf::Vector2f(GRID_SIZE, 0); // Start moving right
    }

    void move() {
        for (int i = body.size() - 1; i > 0; --i)
            body[i].setPosition(body[i - 1].getPosition());
        body[0].setPosition(body[0].getPosition() + direction);
    }

    void grow() {
        body.emplace_back(body.back().getPosition());
    }

    void setDirection(sf::Vector2f newDir) {
        if (body.size() > 1 && body[0].getPosition() + newDir == body[1].getPosition())
            return;
        direction = newDir;
    }

    sf::Vector2f getDirection() {
        return direction;
    }

    sf::Vector2f getHeadPosition() {
        return body[0].getPosition();
    }

    bool checkSelfCollision() {
        for (int i = 1; i < body.size(); ++i)
            if (body[0].getPosition() == body[i].getPosition())
                return true;
        return false;
    }

    void draw(sf::RenderWindow& window) {
        for (auto& segment : body)
            segment.draw(window);
    }

    int getLength() {
        return body.size();
    }
};
```

5. Game: Main Game Logic

```
class Game {
private:
    enum class GameState { Menu, Playing, GameOver };
    GameState state = GameState::Menu;

    sf::RenderWindow window;
    Snake snake;
    Food food;
    sf::Clock clock;
    float moveDelay = 0.2f;
    bool gameOver = false;
    sf::Font font;
    sf::Text gameOverText;
    sf::Text menuText;

    // Menu and GameOver Backgrounds
    sf::RectangleShape menuBackground;
    sf::RectangleShape gameOverBackground;

public:
    Game() : window(sf::VideoMode(WINDOW_WIDTH, WINDOW_HEIGHT), "Snake Game - OOP") {
        std::srand(static_cast<unsigned>(std::time(nullptr)));

        if (!font.loadFromFile("arial.ttf")) {
            throw std::runtime_error("Failed to load font");
        }

        // Menu Background (Gradient)
        menuBackground.setSize(sf::Vector2f(WINDOW_WIDTH, WINDOW_HEIGHT));
        menuBackground.setFillColor(sf::Color(0, 0, 255)); // Blue background
        menuBackground.setPosition(0, 0);

        // Game Over Background
        gameOverBackground.setSize(sf::Vector2f(WINDOW_WIDTH, WINDOW_HEIGHT));
        gameOverBackground.setFillColor(sf::Color(255, 0, 0)); // Red background
        gameOverBackground.setPosition(0, 0);

        // Game Over Text
        gameOverText.setFont(font);
        gameOverText.setString("Game Over!");
        gameOverText.setCharacterSize(50);
        gameOverText.setFillColor(sf::Color::Yellow);
        gameOverText.setPosition(WINDOW_WIDTH / 2 - 150, WINDOW_HEIGHT / 2 - 25);

        // Menu Text
        menuText.setFont(font);
        menuText.setString("Press Enter to Start\nUse Arrow Keys to Move");
        menuText.setCharacterSize(30);
        menuText.setFillColor(sf::Color::White);
        menuText.setPosition(WINDOW_WIDTH / 2 - 200, WINDOW_HEIGHT / 2 - 60);
    }
}
```

```

void run() {
    while (window.isOpen()) {
        processEvents();
        if (state == GameState::Playing && !gameOver)
            update();
        render();
    }
}

private:
void processEvents() {
    sf::Event event;
    while (window.pollEvent(event)) {
        if (event.type == sf::Event::Closed)
            window.close();

        // Start game from menu
        if (state == GameState::Menu && event.type == sf::Event::KeyPressed) {
            if (event.key.code == sf::Keyboard::Enter) {
                state = GameState::Playing;
            }
        }
    }

    if (state == GameState::Playing) {
        if (sf::Keyboard::isKeyPressed(sf::Keyboard::Up) && snake.getDirection().y == 0)
            snake.setDirection(sf::Vector2f(0, -GRID_SIZE));
        else if (sf::Keyboard::isKeyPressed(sf::Keyboard::Down) && snake.getDirection().y == 0)
            snake.setDirection(sf::Vector2f(0, GRID_SIZE));
        else if (sf::Keyboard::isKeyPressed(sf::Keyboard::Left) && snake.getDirection().x == 0)
            snake.setDirection(sf::Vector2f(-GRID_SIZE, 0));
        else if (sf::Keyboard::isKeyPressed(sf::Keyboard::Right) && snake.getDirection().x == 0)
            snake.setDirection(sf::Vector2f(GRID_SIZE, 0));
    }
}

```

```

    }

    void update() {
        if (clock.getElapsedTime().asSeconds() > moveDelay) {
            snake.move();
            clock.restart();

            sf::Vector2f headPos = snake.getHeadPosition();

            // Wall collision
            if (headPos.x < 0 || headPos.x >= WINDOW_WIDTH || headPos.y < 0 || headPos.y >= WINDOW_HEIGHT)
                gameOver = true;

            // Self collision
            if (snake.checkSelfCollision())
                gameOver = true;

            // Food collision
            if (headPos == food.getPosition()) {
                snake.grow();
                food.respawn();
            }
        }
    }

    void render() {
        window.clear();

        // Draw Background
        if (state == GameState::Menu) {
            window.draw(menuBackground);
            window.draw(menuText);
        }
        else if (gameOver) {
            window.draw(gameOverBackground);
            window.draw(gameOverText);
        }
        else {
            window.clear(sf::Color(40, 40, 40)); // Dark gray background
            food.draw(window);
            snake.draw(window);
        }

        window.display();
    }
};

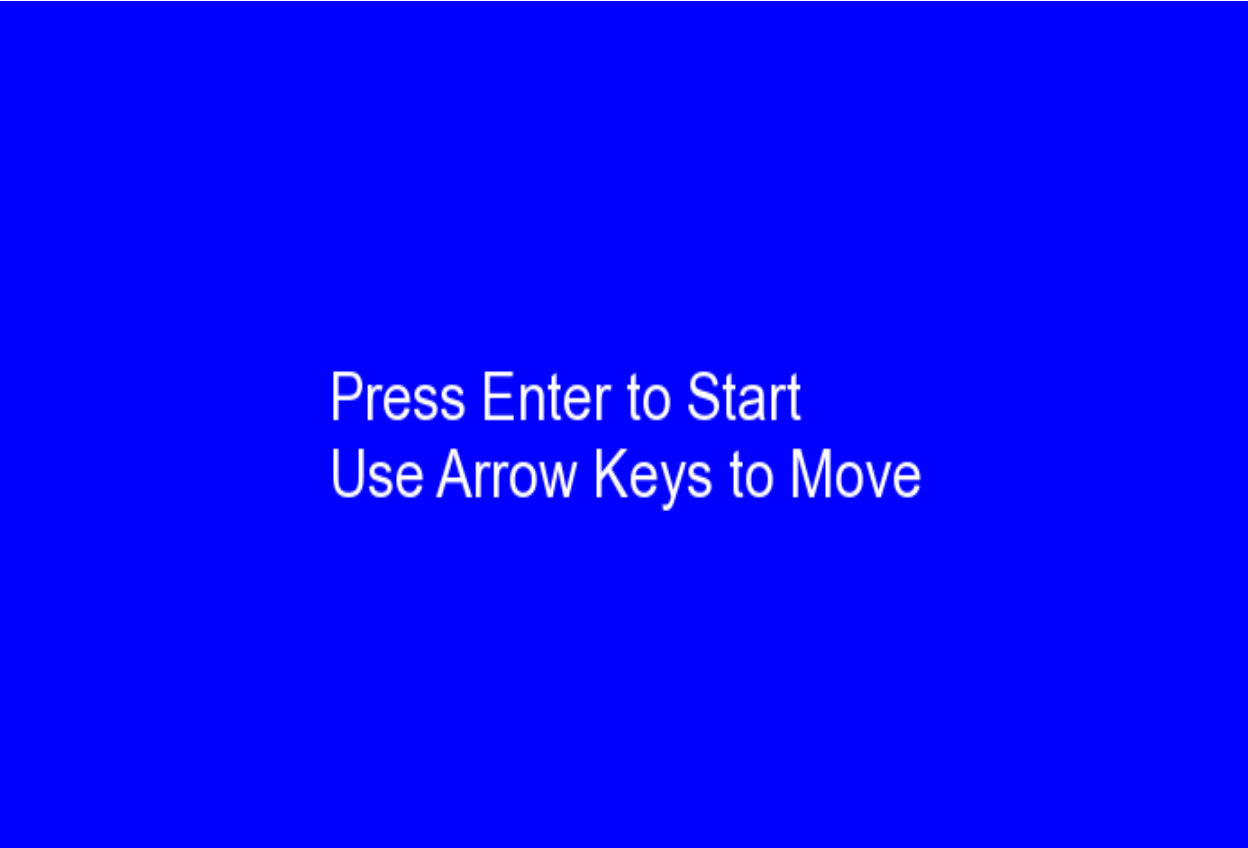
// ----- Main -----
int main() {

```

◆ 7. Sample Input & Output • Screenshots of Running Program:

- Arrow key inputs to control snake.
- Output shows snake eating food and growing.
- Game Over screen on wall or self-collision.

Main Menu:



Press Enter to Start
Use Arrow Keys to Move

Gameplay:



Message if u hit the wall or the snake touches itself:

Game Over!



◆ 8. Results & Discussion

- Was the objective achieved? Yes, the game runs smoothly, and all objectives including OOP principles were implemented successfully.
 - What works well?
 - Smooth movement
 - Accurate collision detection
 - Clear separation of concerns using classes
 - Game terminates when the snake touches itself
 - Any limitations?
 - No scoring system
 - No sound or advanced graphics
-

◆ 9. Conclusion

- Final thoughts: The project effectively combines game logic and OOP principles in C++.
 - What you learned:
 - Applying OOP in real-world projects
 - Using SFML for graphics and setting up SFML which is time consuming.
 - Structuring code for readability and reusability
 - Any future improvements or upgrades:
 - Add score tracking
 - Introduce levels and speed variations
 - Add pause, restart, and more complex and advanced main menu features
-

◆ 10. References (if any)

- <https://www.sfml-dev.org/>
- <https://cplusplus.com/>
- <https://en.cppreference.com/>
- YouTube tutorials on SFML and Snake Game
- ChatGPT for explanation and understanding of key concepts.
